The University of Azad Jammu and Kashmir

Department of Software Engineering

Digital Image Processing Lab

Lab Task # 01 – Python Setup, Fundamentals, and Imports

Student Name	Zulqarnain Mohi Ud Din
Roll Number	2024-SE-07
Semester	Fifth
Session	2024-28 (SE)
Lab Instructor	Engr. Fahad Nisar
Date Assigned / Due	October 16, 2026 / October 23, 2026

Objective

To install Python and VS Code, practice basic Python syntax (variables, conditions, loops, functions and collections), install the introductory packages NumPy, Pillow and Matplotlib, and verify the required import statements.

Part 1: Installation and Setup

Step 1–2: Python and pip versions

Commands run in Command Prompt: python --version and python -m pip –version

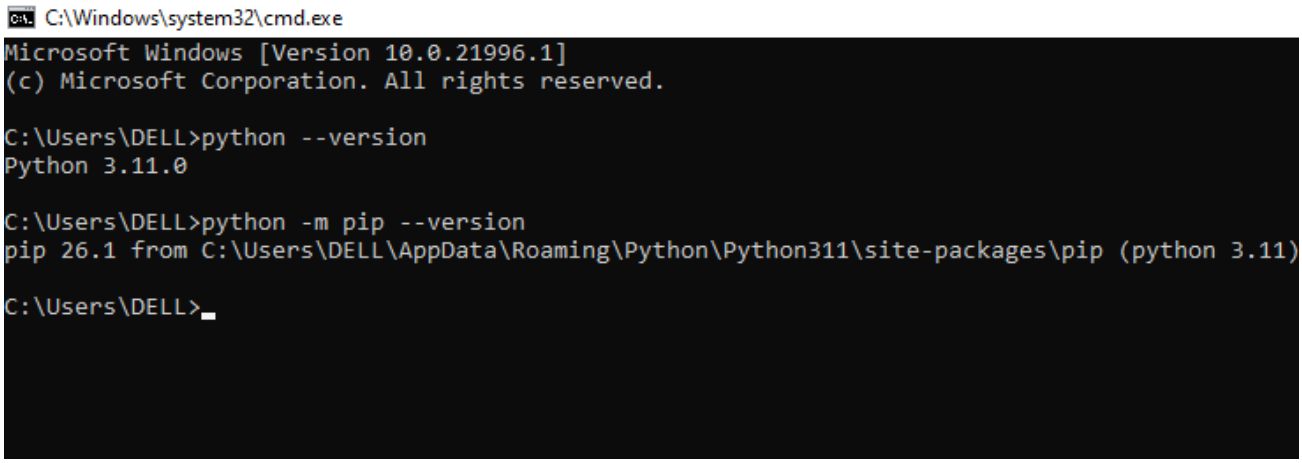


Figure 1: Python and pip version check in Command Prompt

Step 3: VS Code and Python interpreter

Python extension (Microsoft) installed, and the Python 3 interpreter selected via Ctrl + Shift + P → Python: Select Interpreter.

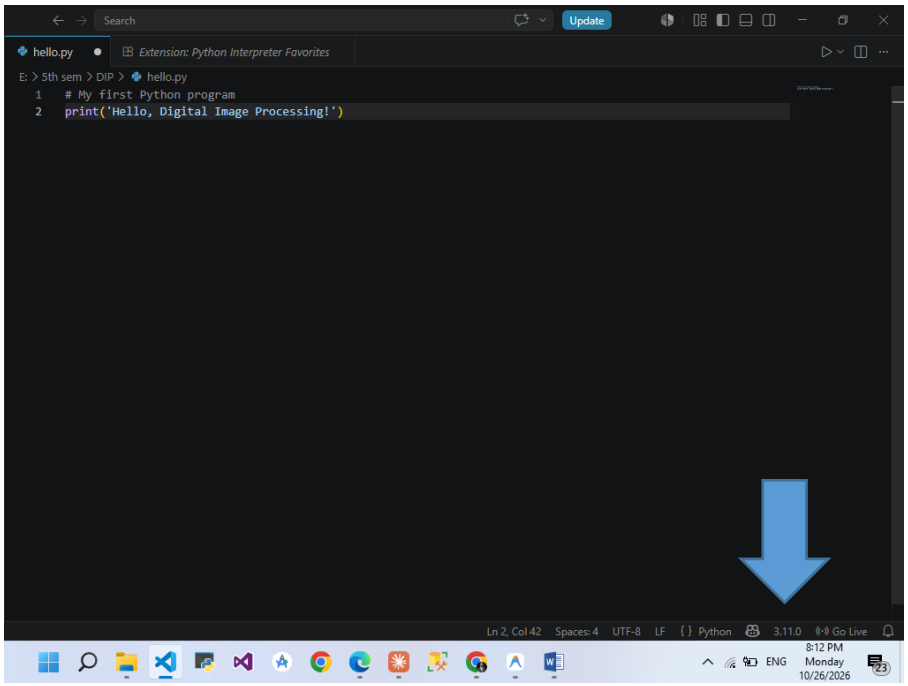


Figure 2: Selected Python interpreter in VS Code

Step 4: First Python program (hello.py)

Code (hello.py)

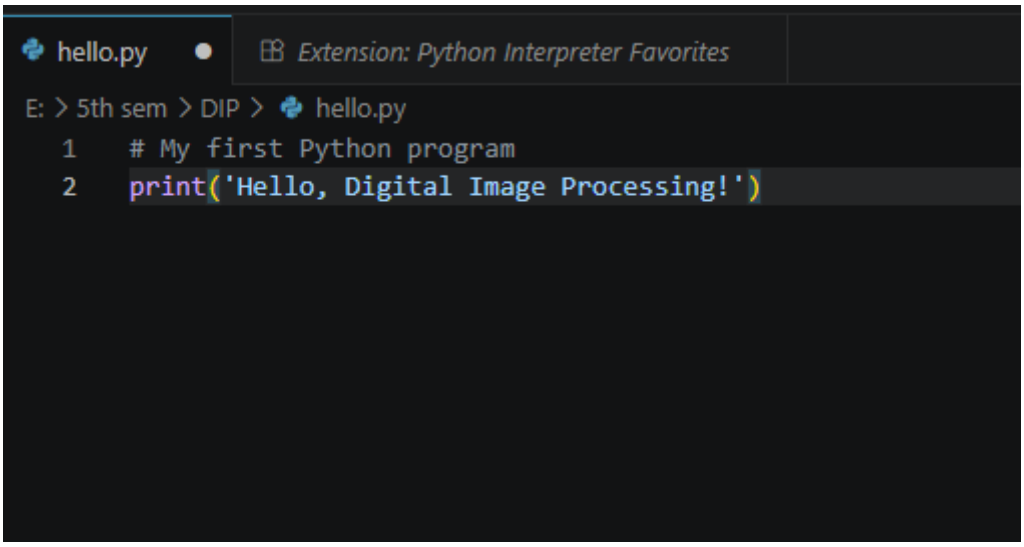


Figure 3: hello.py in VS Code (project folder dip-lab-01 visible)

Output

```
[Running] python -u "e:\5th sem\DIP\hello.py"
Hello, Digital Image Processing!

[Done] exited with code=0 in 0.157 seconds
```

Figure 4: Output of python hello.py

Part 2: Basic Python Exercises (basics.py)

Each exercise is labeled with a comment in basics.py. Run the file with: python basics.py

Exercise 1: Variables and Data Types

Variables width, height, image_name and is_color; print each value and its type; calculate total pixels.

Code

E: > 5th sem > DIP > basics.py > ...

```python
# Lab Task 01: Python Fundamentals
# Digital Image Processing Lab

# ========================================
# Exercise 1: Variables and Data Types
# ========================================
print("--- Exercise 1: Variables and Data Types ---")
width = 640
height = 480
image_name = "sample.png"
is_color = True

print(f"width: {width}, type: {type(width)}")
print(f"height: {height}, type: {type(height)}")
print(f"image_name: {image_name}, type: {type(image_name)}")
print(f"is_color: {is_color}, type: {type(is_color)}")

total_pixels = width * height
print(f"Total number of pixels (width * height): {total_pixels}")
print()

```

Figure 5: Code for Exercise 1

Output

```
[Done] exited with code=0 in 0.157 seconds

[Running] python -u "e:\5th sem\DIP\basics.py"
--- Exercise 1: Variables and Data Types ---
width: 640, type: <class 'int'>
height: 480, type: <class 'int'>
image_name: sample.png, type: <class 'str'>
is_color: True, type: <class 'bool'>
Total number of pixels (width * height): 307200
```

Figure 6: Output for Exercise 1

Exercise 2: if, elif, and else

Label intensities -1, 0, 127, 128, 255 and 256.

Code

```
22  # ========================================
23  # Exercise 2: if, elif, and else
24  # ========================================
25  print("--- Exercise 2: if, elif, and else ---")
26  test_intensities = [-1, 0, 127, 128, 255, 256]
27
28  for intensity in test_intensities:
29      if intensity < 0 or intensity > 255:
30          label = "Invalid"
31      elif intensity >= 128:
32          label = "Bright pixel"
33      else:
34          label = "Dark pixel"
35      print(f"Intensity {intensity}: {label}")
36  print()
37
38  # ========================================
```

Figure 7: Code for Exercise 2

Output

```
--- Exercise 2: if, elif, and else ---
Intensity -1: Invalid
Intensity 0: Dark pixel
Intensity 127: Dark pixel
Intensity 128: Bright pixel
Intensity 255: Bright pixel
Intensity 256: Invalid
```

Figure 8: Output for Exercise 2

Expected: Invalid, Dark pixel, Dark pixel, Bright pixel, Bright pixel, Invalid

Exercise 3: for and while Loops

for loop over [20, 130, 255] with Bright/Dark labels (threshold 128); while loop printing 0, 1, 2.

Code

```
38  # ========================================
39  # Exercise 3: for and while Loops
40  # ========================================
41  print("--- Exercise 3: for and while Loops ---")
42  pixels_list = [20, 130, 255]
43  print("For loop visiting pixel values:")
44  for val in pixels_list:
45      if val >= 128:
46          label = "Bright pixel"
47      else:
48          label = "Dark pixel"
49      print(f"Pixel {val}: {label}")
50
51  print("\nWhile loop printing 0, 1, and 2:")
52  counter = 0
53  while counter < 3:
54      print(counter)
55      counter += 1
56  print()
57
```

Figure 9: Code for Exercise 3

Output

```
--- Exercise 3: for and while Loops ---
For loop visiting pixel values:
Pixel 20: Dark pixel
Pixel 130: Bright pixel
Pixel 255: Bright pixel

While loop printing 0, 1, and 2:
0
1
2
```

Figure 10: Output for Exercise 3

Exercise 4: Functions

Function is_bright(pixel, threshold=128) tested with is_bright(127), is_bright(128), is_bright(200, 210).

Code

```
58  # ========================================
59  # Exercise 4: Functions
60  # ========================================
61  print("--- Exercise 4: Functions ---")
62  def is_bright(pixel, threshold=128):
63      return pixel >= threshold
64
65  print(f"is_bright(127): {is_bright(127)}")
66  print(f"is_bright(128): {is_bright(128)}")
67  print(f"is_bright(200, 210): {is_bright(200, 210)}")
68  print()
```

Figure 11: Code for Exercise 4

Output

```
--- Exercise 4: Functions ---
is_bright(127): False
is_bright(128): True
is_bright(200, 210): False
```

Figure 12: Output for Exercise 4

Exercise 5: Lists and Tuples

List operations on pixels, tuple unpacking of size, and the mutability comment.

Code

```
69
70  # ========================================
71  # Exercise 5: Lists and Tuples
72  # ========================================
73  print("--- Exercise 5: Lists and Tuples ---")
74  pixels = [0, 128, 255]
75  print(f"First item: {pixels[0]}")
76
77  # Replace the middle item (index 1) with 150
78  pixels[1] = 150
79
80  # Append 64
81  pixels.append(64)
82  print(f"Updated list: {pixels}")
83  print(f"Length of list: {len(pixels)}")
84
85  # Create size tuple and unpack into width and height
86  size = (640, 480)
87  w, h = size
88  print(f"Unpacked size -> Width: {w}, Height: {h}")
89
90  # Explanation comment:
91  # In Python, lists are mutable objects, meaning their elements can be modified, added,
92  # or removed in place without altering the object's identity in memory.
93  # In contrast, tuples are immutable data structures designed with fixed values; once created,
94  # their elements and size cannot be reassigned or modified.
95  print()
96
```

Figure 13: Code for Exercise 5

Output

```
--- Exercise 5: Lists and Tuples ---
First item: 0
Updated list: [0, 150, 255, 64]
Length of list: 4
Unpacked size -> Width: 640, Height: 480
```

Figure 14: Output for Exercise 5

Exercise 6: Dictionaries and Sets

Dictionary image with the added key "mode"; set of unique labels.

Code

```
# ========================================
# Exercise 6: Dictionaries and Sets
# ========================================
print("--- Exercise 6: Dictionaries and Sets ---")
image = {"width": 640, "height": 480}
image["mode"] = "RGB"

print(f"Width: {image['width']}")
print(f"Mode: {image['mode']}")

labels = ["cat", "dog", "cat", "bird"]
unique_labels = set(labels)
print(f"Labels set: {unique_labels}")
print(f"Length of unique labels set: {len(unique_labels)}")

```

Figure 15: Code for Exercise 6

Output

```
--- Exercise 6: Dictionaries and Sets ---
Width: 640
Mode: RGB
Labels set: {'cat', 'dog', 'bird'}
Length of unique labels set: 3

[Done] exited with code=0 in 0.319 seconds
```

Figure 16: Output for Exercise 6

Part 3: Required Packages

Step 6: Install NumPy, Pillow and Matplotlib

Command: python -m pip install numpy pillow matplotlib

Confirm installation

Command: python -m pip show numpy pillow matplotlib

```
Microsoft Windows [Version 10.0.21996.1]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DELL>python -m pip show numpy pillow matplotlib
WARNING: Ignoring invalid distribution ~ (C:\Python311\Lib\site-packages)
WARNING: Ignoring invalid distribution ~ip (C:\Python311\Lib\site-packages)
Name: numpy
Version: 2.4.2
Summary: Fundamental package for array computing in Python
Home-page: https://numpy.org
Author: Travis E. Oliphant et al.
Author-email:
License-Expression: BSD-3-Clause AND 0BSD AND MIT AND Zlib AND CC0-1.0
Location: C:\Python311\Lib\site-packages
Requires:
Required-by: contourpy, matplotlib
---
Name: pillow
Version: 12.1.1
Summary: Python Imaging Library (fork)
Home-page: https://python-pillow.github.io
Author:
Author-email: "Jeffrey A. Clark" <aclark@aclark.net>
License-Expression: MIT-CMU
Location: C:\Python311\Lib\site-packages
Requires:
Required-by: matplotlib
---
Name: matplotlib
Version: 3.11.2
Summary: Python plotting package
Home-page: https://matplotlib.org
Author: John D. Hunter, Michael Droettboom
Author-email: Unknown <matplotlib-users@python.org>
License: License agreement for matplotlib versions 1.3.0 and later
```

Figure 17: Package names, versions and locations from pip show

Part 4: Import Statements (lab01_imports.py)

Saved import statements

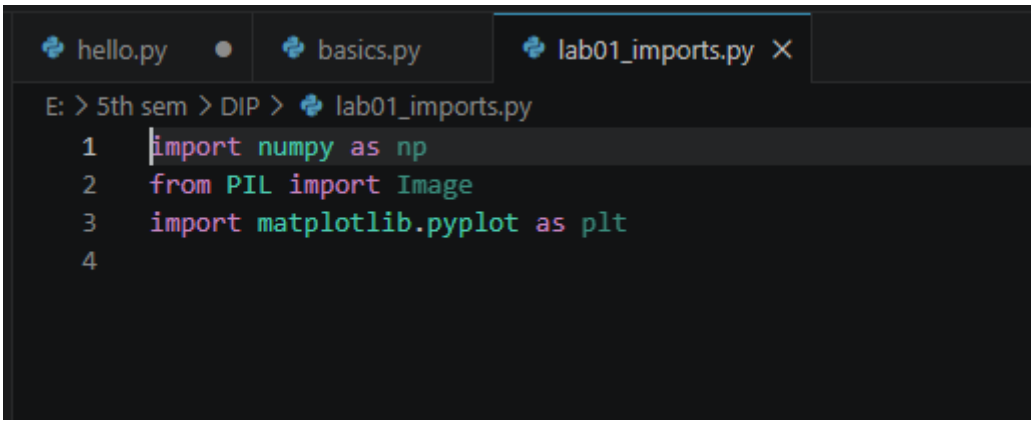


Figure 19: lab01_imports.py containing only the three import lines

Run result

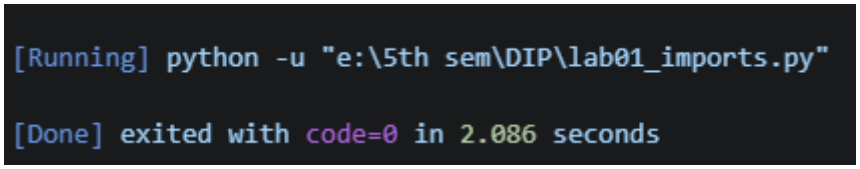


Figure 20: Output of python lab01_imports.py (returns to the prompt with no errors)

Explanation: Python vs pip and Import Statements

Python is the programming language and interpreter that reads and runs our code, while pip is Python's package manager that downloads and installs extra libraries such as NumPy, Pillow and Matplotlib. The import statements load these installed libraries into a program so that their functions (arrays, image handling and plotting) can be used. Running lab01_imports.py without any error confirms that all three packages were installed correctly.

Submitted Files

`hello.py`	First Python program
`basics.py`	Exercises 1–6
`lab01_imports.py`	Import statements for NumPy, Pillow and Matplotlib